Preserving Silent Calls in SPQR Runs
A Two-Agent Definition and Review Experiment

Local research experiment
AI-assisted development and independent agent review

10 November 2026

Abstract

The existing SPQR verification development represents a finite run by its initial state, final state, and list of emitted epoch secrets. Successful calls that emit no key contribute no entry to that list. We introduce a conservative refinement recording one optional output per successful call. We prove that existential erasure of these traces is exactly the existing run relation, that every specified trace boundary admits an intermediate state, and that trace length equals emitted-key count plus silent-call count. A second round defines a prefix epoch clock and proves exact epoch progress at call boundaries and across intervals. The definitions and proofs are checked in Lean against the repository's existing transition relation. A second agent reviews the construction, its trust dependencies, and the limits of its interpretation. This is a small local formalization experiment, with no claim of literature novelty, optimality, or additional cryptographic security.

1 Problem and experimental method

A useful definition exposes the information needed by later proofs without changing the behaviors already modeled. Here the target is the relation Run in the local SPQR verification repository [1]. Its output index records emitted keys in order, but has no position for a successful call producing no key. Induction on a derivation can still reason about such calls; the issue is what information the relation's explicit indices expose, not the absence of silent transitions from the existing semantics.

The user requested two agents: one to improve or introduce definitions and derive theorems, and another to review those results. The author agent produced a separate Lean file and an author report. The reviewer examined the existing semantics, checked the new statements and proofs, and produced a separate review. The coordinating agent assembled this paper and its reproducible artifacts. These roles provide a review workflow, not evidence of statistical independence or a substitute for human review.

We interpret "better" relative to two concrete criteria: retaining successful-call positions and preserving exactly the old relation after erasure. We do not optimize over all possible definitions. Production definitions remain unchanged. All experimental files live under `experiments/two-agent-traces/`, on branch `experiment/two-agent-trace-definitions`.

2 Existing semantics and the refined definition

Let S be the existing type `States` and K the type `EpochSecret`. Write $\mathsf{Step}(s, t, o)$ for the existing relation with $o \in \mathrm{Option}(K)$. It holds when a send or receive call succeeds at both the Aeneas

and Rust result levels, returns state t, and returns optional key o. Send witnesses include RNG interfaces and states; receive witnesses include the message. These witnesses are not explicit indices of our trace either. The relation concerns one party, arbitrary initial states, and successful calls only.

For an optional key o, put $\mathrm{list}(\mathsf{none}) = []$ and $\mathrm{list}(\mathsf{some}(k)) = [k]$. The existing relation has constructors

$$\frac{}{\mathsf{Run}(s, s, [])} \qquad \frac{\mathsf{Step}(s, u, o) \quad \mathsf{Run}(u, t, ks)}{\mathsf{Run}(s, t, \mathrm{list}(o) +\!\!+ ks)}.$$

Consequently, a silent call contributes no element to the emitted-key list.

Definition 1 (Successful-call trace)**.** *A trace is a list* $\tau \in \mathrm{List}(\mathrm{Option}(K))$*. Define the relation* TraceRun *by*

$$\frac{}{\mathsf{TraceRun}(s, s, [])} \qquad \frac{\mathsf{Step}(s, u, o) \quad \mathsf{TraceRun}(u, t, \tau)}{\mathsf{TraceRun}(s, t, o :: \tau)}.$$

Define erasure recursively by $\mathsf{outputs}([]) = []$ *and* $\mathsf{outputs}(o :: \tau) = \mathrm{list}(o) +\!\!+ \mathsf{outputs}(\tau)$.

Every trace entry now corresponds to one successful call, including a silent call. The explicit trace does not distinguish send from receive, record messages, or constrain a sequence of RNG states across calls beyond what the original relation requires. It is an optional-output history, not a full execution log. No uniqueness of intermediate states is asserted.

3 Conservativity and composition

Theorem 1 (Exact recovery of the existing relation)**.** *For all* $s, t \in S$ *and* $ks \in \mathrm{List}(K)$*,*

$$\mathsf{Run}(s, t, ks) \iff \exists \tau,\ \mathsf{TraceRun}(s, t, \tau) \ \wedge\ \mathsf{outputs}(\tau) = ks.$$

Proof. For the forward direction, induct on the old run. The empty run lifts to the empty trace. For a constructor with head output o, lift the tail to τ and prepend o. The erasure equation follows from its recursive definition. For the reverse direction, induct on the trace derivation: each constructor becomes the corresponding old run constructor, whose index prepends $\mathrm{list}(o)$ rather than o. Substitute the supplied erasure equality. □

This is a logical equivalence of relations, with an existential trace witness. It does not assert a unique trace or a canonical executable extraction function from proof terms. It establishes compatibility with every property already stated purely over the old run's endpoints and emitted keys: given such a property for Run, apply it to the erased trace. This observation does not reprove any existing protocol invariant.

Proposition 1 (Erasure and composition)**.** *For all traces* x, y*,* $\mathsf{outputs}(x +\!\!+ y) = \mathsf{outputs}(x) +\!\!+ \mathsf{outputs}(y)$*. If* $\mathsf{TraceRun}(s, u, x)$ *and* $\mathsf{TraceRun}(u, t, y)$*, then* $\mathsf{TraceRun}(s, t, x +\!\!+ y)$*.*

Proof. The erasure equation follows by induction on x and associativity of list concatenation. The relational claim follows by induction on the first trace derivation, retaining each head transition and appending the second derivation at its empty tail. □

4 Exact decomposition at call boundaries

Theorem 2 (Trace splitting)**.** *For all states s, t and traces x, y,*

$$\mathsf{TraceRun}(s, t, x +\!\!+ y) \quad \Longleftrightarrow \quad \exists u,\ \mathsf{TraceRun}(s, u, x) \wedge \mathsf{TraceRun}(u, t, y).$$

Proof. For the forward direction induct on x, generalizing s. If $x = []$, take $u = s$. If $x = o :: x'$, invert the trace constructor to obtain its first transition and remaining trace. Apply the induction hypothesis to that remainder and prepend the first transition to its prefix. The reverse direction is trace composition. □

Theorem 3 (Cut after a specified number of calls)**.** *If $\mathsf{TraceRun}(s, t, \tau)$, then for every natural number n there is a state u such that*

$$\mathsf{TraceRun}(s, u, \mathrm{take}(n, \tau)) \ \wedge \ \mathsf{TraceRun}(u, t, \mathrm{drop}(n, \tau)).$$

Proof. Use $\mathrm{take}(n, \tau) +\!\!+ \mathrm{drop}(n, \tau) = \tau$ and apply trace splitting. □

For $n \leq |\tau|$, the prefix contains exactly n successful calls; for $n > |\tau|$, list operations saturate, giving the whole trace and an empty suffix. The theorem provides existence, not uniqueness, of u. A boundary counts successful calls, not elapsed time, network deliveries, failed calls, or emitted keys.

As a list illustration, take $\tau = [\mathsf{none}, \mathsf{some}(k_1), \mathsf{none}, \mathsf{some}(k_2)]$. Its erasure is $[k_1, k_2]$, and a cut at three calls retains $[\mathsf{none}, \mathsf{some}(k_1), \mathsf{none}]$ as the prefix. If this list has a trace derivation, the cut theorem supplies a corresponding intermediate state. The example itself does not establish that this particular list is realizable by SPQR.

5 Exact accounting and information loss

Definition 2 (Silent-call count)**.** *Define* $\mathsf{silentCount}([]) = 0$, $\mathsf{silentCount}(\mathsf{none} :: \tau) = 1 + \mathsf{silentCount}(\tau)$, *and* $\mathsf{silentCount}(\mathsf{some}(k) :: \tau) = \mathsf{silentCount}(\tau)$.

Theorem 4 (Call accounting)**.** *For every optional-output list τ,*

$$|\tau| = |\mathsf{outputs}(\tau)| + \mathsf{silentCount}(\tau), \qquad |\mathsf{outputs}(\tau)| \leq |\tau|.$$

Proof. Induct on τ and split on its head option. A none increments only the silent count; a some increments only the emitted-key count. Natural-number arithmetic yields the equality, and nonnegativity yields the inequality. □

For a valid trace, this list identity becomes an exact accounting statement about successful calls. It is not a progress bound: neither the definition nor this theorem bounds the number of silent calls needed to produce a key.

Erasure is not injective on unrestricted lists, since $\mathsf{outputs}([]) = \mathsf{outputs}([\mathsf{none}]) = []$ although the two lists differ. This is separately proved in Lean. It does not prove the existence of two realizable traces with the same endpoints and erased outputs. In particular, no silent self-loop is assumed. This distinction is a central constraint from the review.

Result	Lean declaration
Conservativity	`run_iff_exists_trace`
Erasure of a valid trace	`TraceRun.toRun`
Lifting an old run	`Run.exists_trace`
Erasure of concatenation	`outputs_append`
Composition	`TraceRun.append`
Splitting equivalence	`traceRun_append_iff`
Arbitrary call cut	`TraceRun.cut`
Exact call count	`length_eq_outputs_add_silent`
Lower bound on call count	`outputs_length_le`
List erasure noninjectivity	`outputs_not_injective`

Table 1: Correspondence between the paper and checked declarations.

6 Formal artifact and review

All new declarations are in the namespace `spqr.v1.chunked.states.TraceExperiment`. Table 1 maps mathematical results to their Lean declarations; names in the table are relative to that namespace.

The author supplies `TraceRun.lean` and `AUTHOR.md`; the reviewer supplies `REVIEW.md` and separate checks. The review assesses conservativity, successful-call scope, existential rather than unique witnesses, and the difference between arbitrary lists and executable histories. Final commands and trust-audit results are recorded with the artifact so that validation is reproducible rather than inferred from prose.

The pinned repository toolchain is Lean `v4.35.0-rc4`.

Base revision: `e38c4ce87a10c12d709a79962d75ac03ec295182`.

The experiment imports the existing run module and adds no new axioms or admitted proofs. That statement concerns the new development: it does not certify all external cryptographic functions or every assumption elsewhere in the repository. The independent axiom audit reports `propext`, `Classical.choice`, `Quot.sound`, `Aeneas.Std.core.fmt.Formatter`, and 19 inherited private translated `_native.decide.ax_N` axiom constants for the seven run-related results. No audited result depends on `sorryAx`. Pure list results have smaller dependency sets, detailed in the review. Thus these proofs are not axiom-free. The first-round mathematical proofs use the existing step relation parametrically. The second round, below, additionally uses the existing epoch invariant. Neither round proves that a concrete send or receive call succeeds.

7 Round two: an epoch clock at successful-call boundaries

The first round is structural: it treats the transition relation parametrically. The second round connects the refined index to the existing SPQR epoch invariant. Write $F(s) = \mathsf{keyFloor}(s)$ for the epoch that a next emitted key would carry. This is a natural number defined by the existing development, not a new protocol field. For pending states it is the state's epoch; for states that already emitted their epoch's key it is that epoch plus one. The existing theorem `epochs_consecutive` states that an old run emitting ks has consecutive key epochs starting at $F(s)$ and ends with $F(t) = F(s) + |ks|$.

Definition 3 (Prefix epoch clock)**.** *For a trace τ and an initial floor f, define*

$$C(f, \tau, n) = f + |\mathsf{outputs}(\mathrm{take}(n, \tau))|.$$

This clock measures epoch progress after a prefix of successful calls. It counts key emissions rather than physical time. When $n > |\tau|$, its value saturates at the value for the complete trace.

The clock can be computed from the initial floor and optional-output list without choosing intermediate states. Relating it to the actual state requires a valid trace hypothesis and the existing epoch invariant. Neither the clock definition nor the invariant guarantees that a key-emitting call ever succeeds.

Theorem 5 (Epoch at a call boundary)**.** *If* $\mathsf{TraceRun}(s, t, \tau)$*, then for every* n *there exists* u *such that*

$$\mathsf{TraceRun}(s, u, \mathrm{take}(n, \tau)) \ \wedge \ \mathsf{TraceRun}(u, t, \mathrm{drop}(n, \tau)) \ \wedge \ F(u) = C(F(s), \tau, n).$$

Moreover, if $\mathsf{TraceRun}(s, u, p)$ *and* $\mathsf{TraceRun}(s, v, p)$ *then* $F(u) = F(v)$*.*

Proof. The structural cut theorem supplies a prefix and suffix. Apply the existing endpoint identity to the erased prefix to obtain $F(u) = F(s) + |\mathsf{outputs}(\mathrm{take}(n, \tau))|$. For the second assertion, both floors equal $F(s) + |\mathsf{outputs}(p)|$. □

The uniqueness assertion concerns the floor only, not the full intermediate state.

Theorem 6 (Epoch of a positioned emission)**.** *If* $\mathsf{TraceRun}(s, t, p +\!\!+ (\mathsf{some}(k) :: q))$*, then*

$$k.\mathsf{epoch} = F(s) + |\mathsf{outputs}(p)| = C(F(s), p +\!\!+ (\mathsf{some}(k) :: q), |p|),$$

where the epoch on the left is interpreted as its natural-number value. The emitting call has zero-based position $|p|$*, and this clock is its value immediately before that call.*

Proof. Split the trace after p and invert the first constructor of its suffix. The existing one-step theorem `Step.keyFloor_step` identifies the emitted epoch with the intermediate floor. Prefix endpoint accounting gives the first equality; taking exactly $|p|$ entries of the full trace gives the second. □

Theorem 7 (Exact interval progress and silence)**.** *For every state* s *and lists* p, m, q*, put* $\tau = p +\!\!+ m +\!\!+ q$*. Then*

$$C(F(s), \tau, |p| + |m|) = C(F(s), \tau, |p|) + |\mathsf{outputs}(m)|.$$

The two boundary clocks are equal if and only if every entry of m *is* none*. If* $\mathsf{TraceRun}(s, t, \tau)$*, then* $F(t) = F(s)$ *if and only if every entry of* τ *is* none*.*

Proof. The two relevant prefixes are p and $p +\!\!+ m$. Erasure respects concatenation, so their output lengths differ by $|\mathsf{outputs}(m)|$. Cancellation in the natural numbers shows equality exactly when this length is zero. Induction on a list establishes that erasure is empty precisely when every entry is none. For the final assertion, apply the same reasoning to the endpoint identity for a valid trace. □

The interval clock identities hold for arbitrary lists; their interpretation as actual state progress requires trace validity. An unchanged floor does not mean that the full state is unchanged. Likewise, a silent interval need not be short, and the theorem supplies no time bound or eventual-emission guarantee.

The Lean definition `epochClock` takes an initial state s rather than the explicit natural-number parameter f used above; its value is exactly $C(F(s), \tau, n)$. The new file imports the unchanged

Round-two result	Lean declaration
Endpoint epoch accounting	`TraceRun.keyFloor_eq`
Clock at a cut	`TraceRun.cut_epochClock`
Unique boundary floor	`TraceRun.boundary_keyFloor_unique`
Positioned emission	`TraceRun.emission_epoch`
Prefix clock identity	`epochClock_prefix`
Emission matches clock	`TraceRun.emission_epochClock`
Interval accounting	`epochClock_interval`
Empty erasure characterization	`outputs_eq_nil_iff`
Unchanged interval clock	`epochClock_interval_eq_iff`
Unchanged endpoint floor	`TraceRun.keyFloor_eq_iff_silent`

Table 2: The ten round-two theorems in `EpochTrace.lean`.

first-round module and the repository's existing `KeyFloor` module. These results are derived from that existing invariant; they do not independently re-establish its per-transition case analysis.

The second reviewer round independently checked all ten new theorems, boundary and saturation examples, and their axiom dependencies. The six protocol-connected results have exactly the same 23 reported axiom constants as the existing `epochs_consecutive` and `Step.keyFloor_step`; the other four results have only subsets of the standard axioms. No result depends on `sorryAx`, and no new axioms were introduced. Detailed checks and findings are recorded in `ROUND2_REVIEW.md`. The first-round source and historical review remain unchanged.

8 Limitations and outcome

This experiment establishes a small refinement suited to reasoning at successful-call boundaries. The old definition remains useful for properties depending only on emitted keys. The refinement costs an additional trace index and supporting lemmas, and does not supply liveness, computational security, two-party agreement, or resource bounds. It does not record failed calls or explicit action labels. A richer action trace would require a separate definition and another conservativity argument.

The new results are additions to this experiment, not claims of mathematical novelty in the literature. They use elementary inductive arguments, now checked against the concrete repository relation. The review narrows their interpretation and documents what would require further evidence. No global "best definition" claim follows from the experiment.

References

[1] SPQR verification repository, local base revision cited above.
`Spqr/Specs/Braid/StateMachine/Run.lean`. Existing definitions of `Step`, `Run`, and run composition.

[2] Two-agent trace experiment, accompanying local artifacts.
`experiments/two-agent-traces/TraceRun.lean`, `AUTHOR.md`, `REVIEW.md`, `EpochTrace.lean`, `ROUND2_AUTHOR.md`, and `ROUND2_REVIEW.md`, 2026.